

Constraints

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input



Sample output



3

1

2

0

3

1

4

Explanation

For test case 1:

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

For test case 2:

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3:

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1, as only one operation is required to convert given number



Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains T denoting the total number of test cases.
- The first line of each test case contains an integer N denoting the number that must be converted into a binary palindromic number.

Output format

For each test case in a new line, print the minimum number of operations that are required to convert a number into a binary palindromic number.

Constraints

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input

3
2
3
4



Sample output

1
0
1



Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains T denoting the total number of test cases.
- The first line of each test case contains an integer N denoting the number that must be converted into a binary palindromic number.

Output format

For each test case in a new line, print the minimum number of operations that are required to convert a number into a binary palindromic number.

Constraints

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input

3
2
3
4



Sample output



1
0
1

Question 3

Max. score: 100.00

Binary palindromic numbers

Given a number N . You are required to convert it into a binary palindromic number. A binary palindromic number is a number whose binary representation is a palindrome.

You can perform the following two operations on the provided number:

- Increase the value of the number by 1.
- Decrease the value of the number by 1.

You are required to calculate the minimum number of operations required to convert the given number into a binary palindromic number.

Note: The given number must be represented in the form of the minimum number of bits (ignoring the leading zeros in its binary representation).

Function description

Complete the *solve* function. This function takes the following parameter and returns the minimum number of operations required.

Parameter:

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input



Sample output



3

2

3

4

1

0

1

Explanation

For test case 1.

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

For test case 2.

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3.

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input

3
2
3
4

Sample output

1
0
1

Explanation

For test case 1:

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

For test case 2:

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3:

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input



Sample output

3
2
3
41
0
1

Explanation

For test case 1:

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

For test case 2:

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3:

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

[Previous Question](#)[Next Question](#)

Explanation

For test case 1:

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

For test case 2:

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3:

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 1.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

C++, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml,

[Previous Question](#)

[Next Question](#)

Explanation

For test case 1:

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

For test case 2:

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3:

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1, as only one operation is required to convert given number into binary palindromic number.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 1.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml,

[< Previous Question](#)

[Next Question >](#)

Explanation

For test case 1

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1 as only one operation is required to convert given number into binary palindromic number.

For test case 2

A binary representation of 3 is 11, which is already palindrome. So, answer is 0 since no operations are required.

For test case 3

A binary representation of 4 is 100, which is not palindromic. So, on incrementing 4 by one, we get 5 which is palindromic in binary representation 101. So answer is 1 as only one operation is required to convert given number into binary palindromic number.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 10 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml,

operations that are required to convert a number into a binary palindromic number.

Constraints

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input

3

2

3

4



Sample output

1

0

1

Explanation

For test case 1:

A binary representation of 2 is 10, which is not palindromic. So, on incrementing 2 by one, we get 3 which is palindromic in binary representation. So answer is 1, as only one operation is required to convert given number into a binary palindromic number.

For test case 2:

A binary representation of 3 is 11, which is already a palindrome. So, answer is 0, since no operations are required.

For test case 3:

A binary representation of 4 is 100, which is not palindromic. So, on

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains T denoting the total number of test cases.
- The first line of each test case contains an integer N denoting the number that must be converted into a binary palindromic number.

Output format

For each test case in a new line, print the minimum number of operations that are required to convert a number into a binary palindromic number.

Constraints

$$1 \leq T \leq 10^5$$

$$1 \leq N \leq 2 \times 10^9$$

Sample input



Sample output

3

1

2

0

3

1

4

Binary palindromic numbers

Given a number N . You are required to convert it into a binary palindromic number. A binary palindromic number is a number whose binary representation is a palindrome.

You can perform the following two operations on the provided number:

- Increase the value of the number by 1.
- Decrease the value of the number by 1.

You are required to calculate the minimum number of operations required to convert the given number into a binary palindromic number.

Note: The given number must be represented in the form of the minimum number of bits (ignoring the leading zeros in its binary representation).

Function description

Complete the *solve* function. This function takes the following parameter and returns the minimum number of operations required.

Parameter:

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

Binary palindromic numbers

Given a number N . You are required to convert it into a binary palindromic number. A binary palindromic number is a number whose binary representation is a palindrome.

You can perform the following two operations on the provided number:

- Increase the value of the number by 1.
- Decrease the value of the number by 1.

You are required to calculate the minimum number of operations required to convert the given number into a binary palindromic number.

Note: The given number must be represented in the form of the minimum number of bits (ignoring the leading zeros in its binary representation).

Function description

Complete the *solve* function. This function takes the following parameter and returns the minimum number of operations required.

Parameter:

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

Binary palindromic numbers

Given a number N . You are required to convert it into a binary palindromic number. A binary palindromic number is a number whose binary representation is a palindrome.

You can perform the following two operations on the provided number:

- Increase the value of the number by 1.
- Decrease the value of the number by 1.

You are required to calculate the minimum number of operations required to convert the given number into a binary palindromic number.

Note: The given number must be represented in the form of the minimum number of bits (ignoring the leading zeros in its binary representation).

Function description

Complete the *solve* function. This function takes the following parameter and returns the minimum number of operations required.

Parameter:

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

Binary palindromic numbers

Given a number N . You are required to convert it into a binary palindromic number. A binary palindromic number is a number whose binary representation is a palindrome.

You can perform the following two operations on the provided number:

- Increase the value of the number by 1.
- Decrease the value of the number by 1.

You are required to calculate the minimum number of operations required to convert the given number into a binary palindromic number.

Note: The given number must be represented in the form of the minimum number of bits (ignoring the leading zeros in its binary representation).

Function description

Complete the *solve* function. This function takes the following parameter and returns the minimum number of operations required.

Parameter:

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

Input format for custom testing

Binary palindromic numbers

Given a number N . You are required to convert it into a binary palindromic number. A binary palindromic number is a number whose binary representation is a palindrome.

You can perform the following two operations on the provided number:

- Increase the value of the number by **1**.
- Decrease the value of the number by **1**.

You are required to calculate the minimum number of operations required to convert the given number into a binary palindromic number.

Note: The given number must be represented in the form of the minimum number of bits (ignoring the leading zeros in its binary representation).

Function description

Complete the *solve* function. This function takes the following parameter and returns the minimum number of operations required.

Parameter:

- *num*: Represents the number N , on which you must use a minimum number of operations, to convert the given number into a binary palindromic number.

Input format for custom testing

[Previous Question](#)[Next Question](#)

Crayon Removal Scoring Game

You have a container filled with crayons of various colors, where each color is represented by a different alphabet letter. In each move, you can eliminate a sequence of adjacent crayons that share the same color. You may repeat this process multiple times until the container becomes empty. For each move, you must select a continuous group of identical colored crayons (group of x crayons, where $x \geq 1$). Upon removing them, you earn $x \times x$ points.

You are provided with an integer **N** representing the total count of crayons in the container. You also receive an array **colors** containing the **N** colors in the container, where each color is denoted by an English alphabet letter.

Calculate the maximum points you can achieve in this scenario.

Function description

Complete the **solve()** function. This function takes the following 2 arguments and returns the answer.

Parameters:

- **N**: Represents the total crayons in the box
- **colors[]**: Represents the array of colors of crayons in the box. Each color is represented by an English alphabet.

Input format for custom testing

[< Previous Question](#)[Next Question >](#)

Explanation

- Initially, Points = 0.
- In operation 1, remove crayon set of color b from index 2 to 4. colors = [a, a, a, a, c]. Points = $0 + 2 \times 2 = 4$.
- In operation 2, remove crayon set of color c at index 4. colors = [a, a, a, a]. Points = $4 + 1 \times 1 = 5$.
- In operation 3, remove crayon set of color a from index 0 to 3. colors = []. Points = $5 + 4 \times 4 = 21$.
- Hence, the maximum point is 21.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file
Memory Limit: 256 MB
Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, JavaScript, Kotlin, Lua, MATLAB, OCaml, Perl, PHP, Python 2, Python 3, R, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic

[< Previous Question](#)[Next Question >](#)

080 ▶

F1

F2

F3

(F3)

F4

Z

F5

F5

F6

F6

- **N**: Represents the total crayons in the box
- **colors[]**: Represents the array of colors of crayons in the box. Each color is represented by an English alphabet.

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains an integer **N** denoting the total number of crayons in the box.
- The second line contains **N** space-separated characters denoting the color of the crayons in the box.

Output format

Return the maximum number of points you can get in the given scenario.

Constraints

$$1 \leq N \leq 60$$

colors contain only lowercase English letters.

Sample input

```
7
a a b b a a c
```

Sample output

```
21
```

[< Previous Question](#)[Next Question >](#)

1080

F1

F2

F3



F4

Z²

F5



F6



Crayon Removal Scoring Game

You have a container filled with crayons of various colors, where each color is represented by a different alphabet letter. In each move, you can eliminate a sequence of adjacent crayons that share the same color. You may repeat this process multiple times until the container becomes empty. For each move, you must select a continuous group of identical colored crayons (group of x crayons, where $x \geq 1$). Upon removing them, you earn $x*x$ points.

You are provided with an integer **N** representing the total count of crayons in the container. You also receive an array **colors** containing the **N** colors in the container, where each color is denoted by an English alphabet letter.

Calculate the maximum points you can achieve in this scenario.

Function description

Complete the **solve()** function. This function takes the following 2 arguments and returns the answer.

Parameters:

- **N**: Represents the total crayons in the box
- **colors[]**: Represents the array of colors of crayons in the box. Each color is represented by an English alphabet.

Input format for custom testing

[< Previous Question](#)[Next Question >](#)

◀ 1080 ▶

F1

F2

F3

(())

F4

Z

F5

□ □

F6

+ -

@

#

\$

%

^

Crayon Removal Scoring Game

You have a container filled with crayons of various colors, where each color is represented by a different alphabet letter. In each move, you can eliminate a sequence of adjacent crayons that share the same color. You may repeat this process multiple times until the container becomes empty. For each move, you must select a continuous group of identical colored crayons (group of x crayons, where $x \geq 1$). Upon removing them, you earn $x \times x$ points.

You are provided with an integer **N** representing the total count of crayons in the container. You also receive an array **colors** containing the **N** colors in the container, where each color is denoted by an English alphabet letter.

Calculate the maximum points you can achieve in this scenario.

Function description

Complete the **solve()** function. This function takes the following 2 arguments and returns the answer.

Parameters:

- **N**: Represents the total crayons in the box
- **colors[]**: Represents the array of colors of crayons in the box. Each color is represented by an English alphabet.

Input format for custom testing

[< Previous Question](#)[Next Question >](#)

◀ 1080 ▶

F1

F2

F3

(())

F4

Z

F5

□ □

F6

☑

@

#

\$

%

^